



AIRFLOW SUMMIT 2026

From Broken DAGs to Self-Healing Pipelines

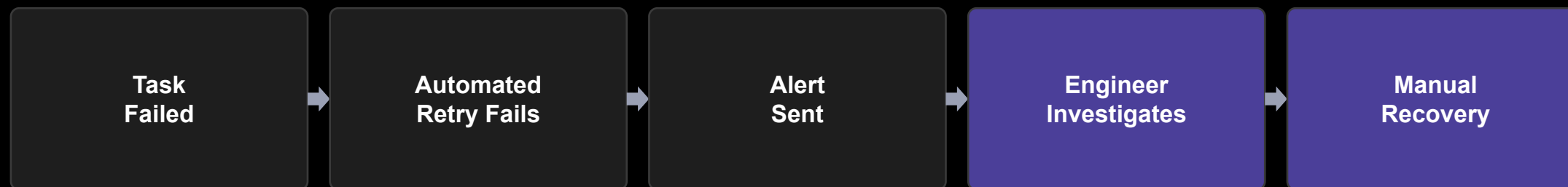
AI-Driven Orchestration with Airflow

Naga Durga Rao Dindi

Data Engineer, Toyota

The 2:00 AM Incident Story

A familiar pattern for anyone running production pipelines



Detection and retries are automated.

Diagnosis and recovery are still manual.

Why Data Pipelines Fail



Upstream Failures

- Missing files
- Late-arriving data
- Unavailable APIs



Data Issues

- Schema drift
- Null spikes
- Duplicate records



Infrastructure Issues

- Worker failures
- Network outages
- Service interruptions



Volume Spikes

- Unexpected data growth
- Processing bottlenecks
- Skewed workloads



Resource Exhaustion

- Out-of-memory errors
- CPU saturation
- Storage pressure



Configuration Problems

- Invalid parameters
- Dependency changes
- Credential misconfiguration

A failed task is a symptom, not necessarily the root cause

Why Retries Are Not Enough?

Failure Type	Does Retry Work?
Timeout	YES
Missing File	NO
Schema Drift	NO
Credential Issue	NO
Memory Exhaustion	⚠️ NOT ALWAYS

Key Message: Different failures require different recovery strategies

What Does Self-Healing Really Mean?



A pipeline is not self-healing until it can observe, diagnose, recover and verify outcomes.

Signals & Evidence



Airflow Signals

- DAG state
- Task runtime
- Retry counts



Data Signals

- Row count
- Freshness
- Schema



Infra Signals

- CPU
- Memory
- Network



Historical Signals

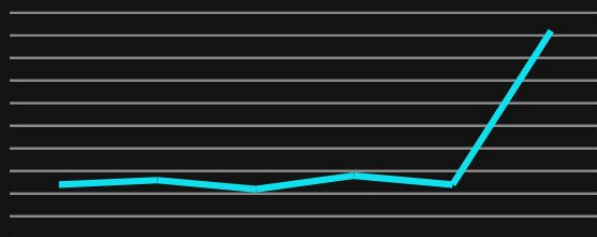
- Previous successful runs
- Runtime baselines
- Incident trends

AI needs evidence, not just logs

Detecting Problems Before Failures

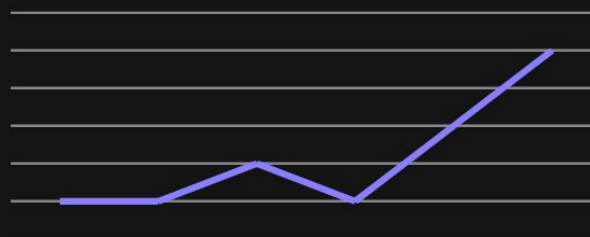
Leading indicators an anomaly model can catch early

Runtime Spike



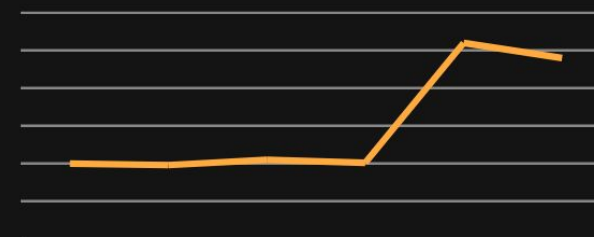
A task that normally finishes in minutes starts creeping toward hours

Retry Spike



Retry counts trend upward across runs before a hard failure occurs

Data Volume Spike



Row counts or payload size jump well outside the historical range

Early signals often appear before hard failures.

Introducing AIRA

AIRA

*AI-Driven Recovery Agent
for Airflow*

A conceptual agentic reliability architecture that combines AI reasoning, retrieval, memory, controlled tool access, deterministic policy, and Apache Airflow orchestration.



Monitor Signals



Detect Incidents



Investigate Causes



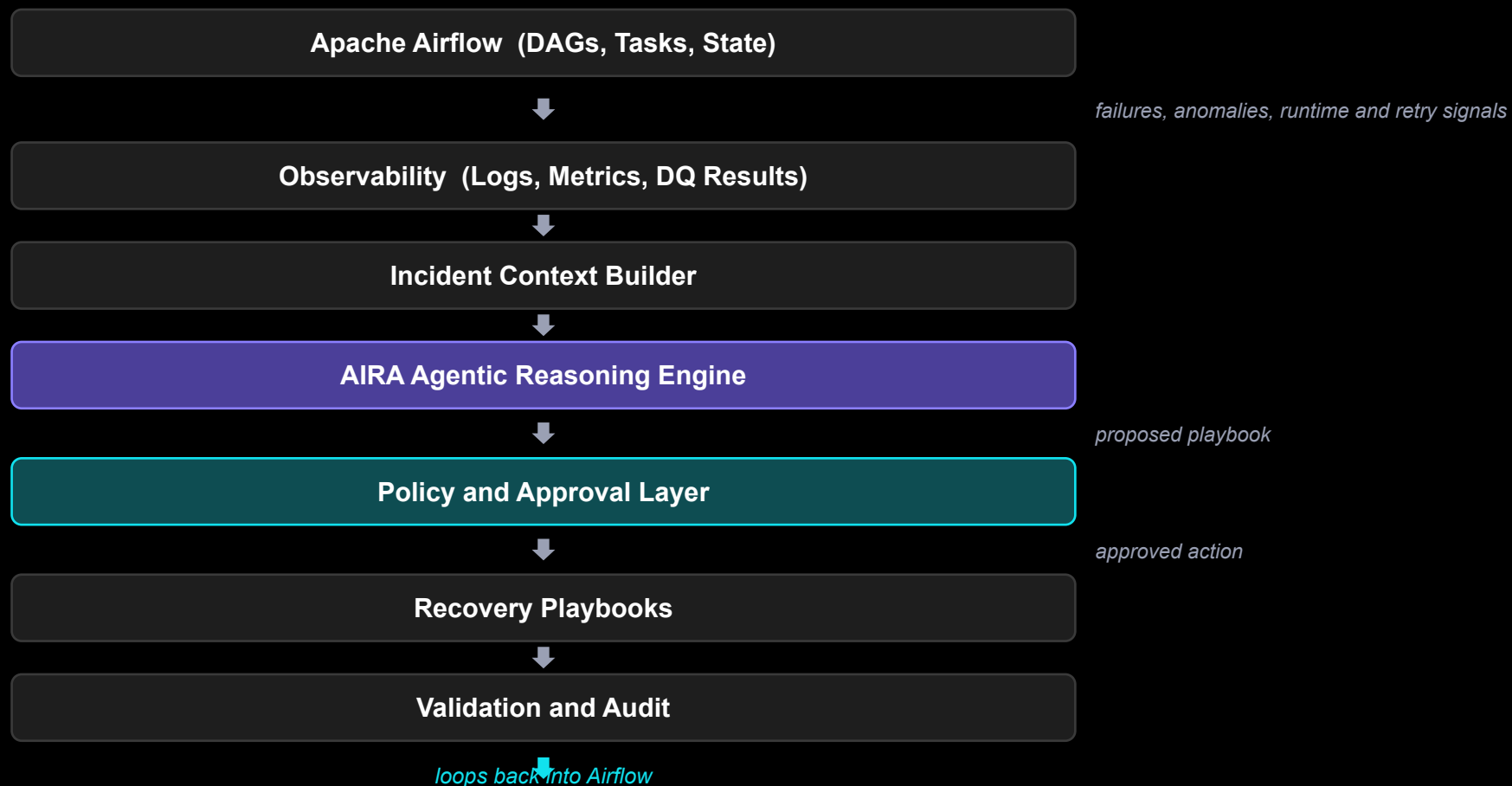
Recommend Recovery



Validate Outcomes

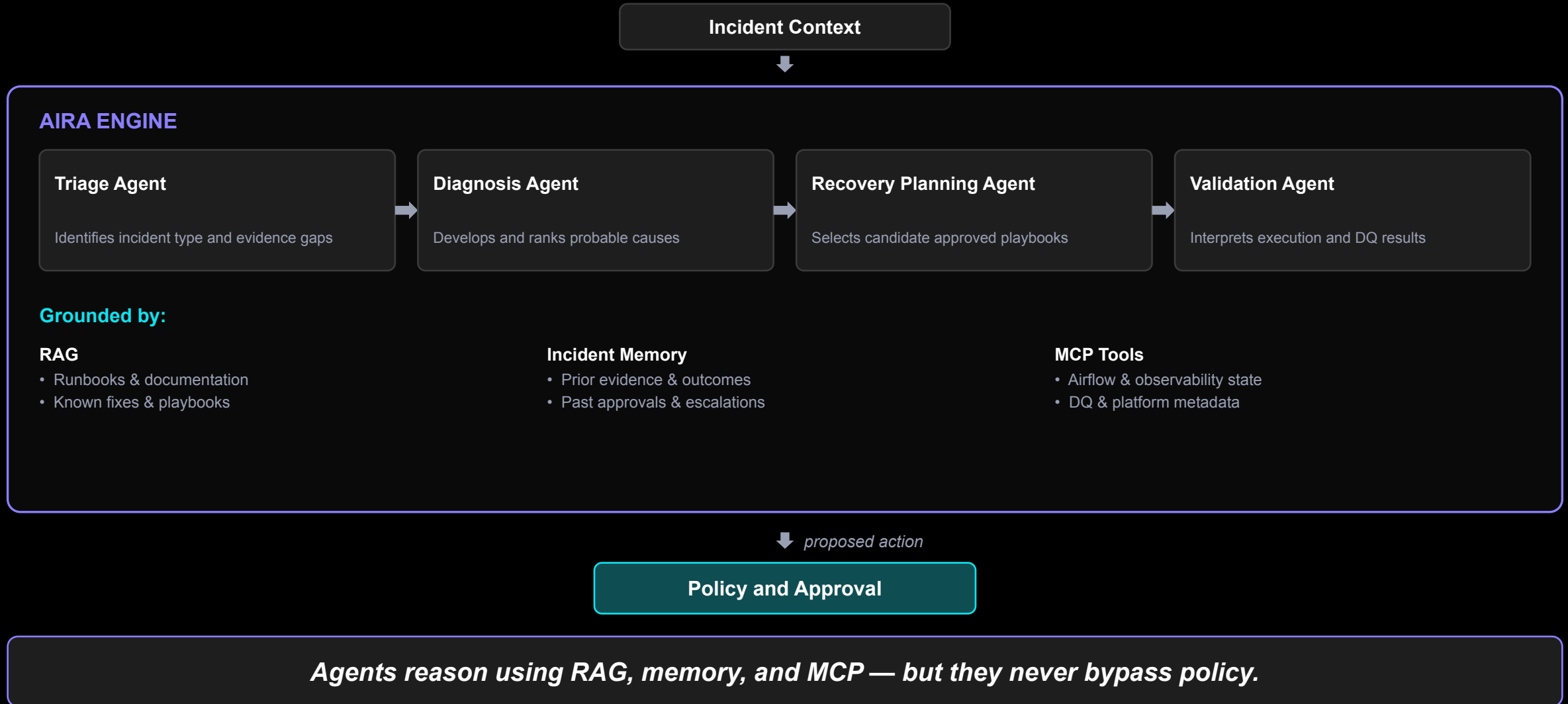
AI reasons. Policy governs. Airflow orchestrates.

AIRA Reference Architecture



Signals flow into AIRA. Only approved recovery actions flow back into Airflow.

Inside the AIRA Agentic Reasoning Engine



Use AI for Ambiguity. Use Deterministic Controls for Authority.

AI Agents

- Correlate distributed evidence
- Summarize incidents
- Generate and rank hypotheses
- Retrieve relevant runbooks
- Recommend playbooks
- Explain uncertainty

Deterministic Components

- Validate tool parameters
- Enforce action allowlists
- Check retry limits
- Check idempotency metadata
- Enforce permissions
- Require approval
- Execute approved playbooks
- Run data-quality rules

AI proposes. Policy authorizes. Automation executes. Validation proves.

Agents Select Playbooks. They Do Not Invent Production Actions.

Failure Pattern	Candidate Playbook	Decision
Temporary API outage	Bounded retry with backoff	May auto-execute
Late source file	Wait, recheck, then retry	May auto-execute
Approved type mismatch	Apply validated cast	Policy-dependent
Capacity pressure	Use approved resource profile	Approval may be required
Schema semantic change	Quarantine and escalate	Human required
Credential or permission issue	Escalate	No autonomous change
Unknown failure	Preserve evidence and escalate	No action

One Incident Across the Complete AIRA Loop

1

Detect

Runtime and volume exceed historical range

2

Triage Agent

Classifies the incident as resource or data related

3

MCP Evidence Gathering

Retrieves Airflow status, memory usage, volume metrics and partition information

4

RAG and Memory

Retrieves approved resource runbook and similar resolved incidents

5

Diagnosis Agent

Ranks volume-driven exhaustion above infrastructure instability

6

Recovery Planning Agent

Recommends an approved response

7

Policy

Checks cost boundary, task idempotency, scope and approval requirement

8

Recovery

Executes only the approved playbook

9

Validation Agent

Reviews task and DQ results

10

Memory and Audit

Records the incident and outcome

Agentic Does Not Mean Unrestricted

Guardrails

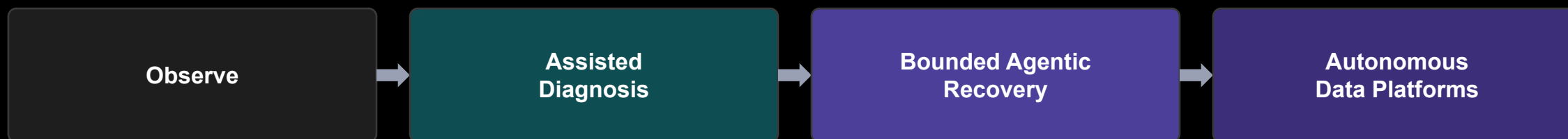
- Read-only evidence tools by default
- Least-privilege identities
- Typed MCP tool inputs
- Approved recovery playbooks
- Bounded scope and retry attempts
- Human approval for medium-risk actions
- Escalation for unknown or high-impact incidents
- Complete audit records

Lessons Learned

- Context quality matters more than model size
- RAG does not guarantee correctness
- Similar incidents are evidence, not proof
- Agents should expose uncertainty
- Native platform resilience should be used first
- Validation is required before resolution
- Unknown is a safe and legitimate result

The safest recovery is sometimes to stop and escalate.

From Assisted Diagnosis to Bounded Agentic Recovery



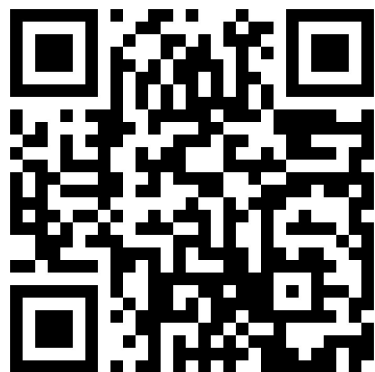
Key Takeaways

- ✓ Retries $\neq$ Diagnosis
- ✓ AI needs evidence
- ✓ Governance is mandatory
- ✓ Verification matters
- ✓ Start small, automate gradually

Questions?

Thank you for coming — let's talk pipelines.

SCAN TO CONNECT



GitHub

github.com/Durga429/aira



LinkedIn

linkedin.com/in/durga29